

Actions

As mentioned earlier, action statements are enclosed within {}. They are similar to the assignment, conditional and looping statements of the C language. A statement may end with a newline character or ";". Comments can be specified using a #.

Variables

The man page states that variables in Awk are dynamic in nature, i.e., they come into existence when they are used. Variable values are either strings or floating-point numbers. Their type is decided based on the context they are used in. They can be assigned values as *variable=value* and can be used in expressions and/or statements using their names. For example, *var=25; print var*3* will output 75. In this case, the variable *var* is treated as a floating-point number.

Besides the RS, FS and ORS variables that we covered earlier, there are a few others that are very useful while writing programs in Awk.

- **NF:** The NF variable provides the number of fields there are in the current input record. If the current record has six fields, then the value of NF would be 6. This is very useful for looping through the fields of a record when the number of fields is not known. *\$NF* represents the last field in a record.
- **NR:** The NR variable provides the number of input records that have so far been read.
- **FNR:** This variable provides the number of input records that have so far been read for the current input file. FNR and NR will be same if only one input file is provided. However, if there are more than one input files, FNR will show the number of records read in the current input file, whereas, NR will show the number of records read since the first record of the first input file.
- **FILENAME:** This provides the name of the current input file. If the input is specified on the command line, then its value is "-".
- **IGNORECASE:** By default, all string operations and regular-expression matches in Awk are case-sensitive. To change that, IGNORECASE needs to be set a value greater than 0.
- **OFS:** The OFS variable specifies the field separator to be used in the output. By default, it's a space character.
- **Operators:** Operators in Awk are similar to various programming languages. Like =, +=, -= etc for assignment. Logical operators &&, ||, ! and the increment(++), decrement(--), operators. One notable exception is the ~ operator. This operator is used to match against a regular expression. It can be negated using a ! operator to check against a negative match. Operators have a particular order of precedence. Please check the Awk man page for reference.

Control statements

Awk provides a variety of control statements that are similar to the control statements used in C. They start with keywords like *if*, *for*, *while*, *continue*, etc. This differentiates them from the simple expressions. Awk provides both the varieties of the *for* loop.

```
for (i = 1; i <= var; i++)
    print i
as well as
```

```
for (i in array)
    print array[i]
```

The second form is very useful for looping through the elements of an array, whose indices are string values. We will cover the arrays and some other advanced features of Awk in the next part of this article.

Passing variables to Gawk

Gawk supports the -v switch that allows users to pass variables to an Awk program. Shell variables can also be passed on to the Gawk program using the -v switch. The syntax is:

```
awk -v varname=${shell_var} 'pattern{ action}'
```

Once defined, *varname* can be used like any dynamic variable defined in Gawk.

Functions

There are several built-in functions available in Gawk. Users can also define their own functions and use them. We will get into the details of various functions available and how to define our own functions in the next article in this series on Awk. For now, we will take a look at a few important functions.

- **print:** As the name suggests, the *print* function allows printing of text/values. If called without any arguments, it prints the current record. Thus, *print* and *print \$0* are equivalent. Its output can be redirected to a file using > *filename*.
- **length:** The *length(str)* function returns the length of the string *str* or the length of *\$0* if no *str* is provided.
- **gsub:** The *gsub* function allows you to replace every occurrence of the text matching a pattern *ptr*, with a substitute string *str* in the target string *tr*. Its syntax is *gsub(ptr, str, tr)*. If no target string is provided, *\$0* is used. *gsub* returns the number of substitutions that took place.
- **sub:** The *sub* function is similar to *gsub*, but it replaces only the first occurrence of the pattern.

Some examples

We will make a copy of the file */etc/passwd* as *passwd2* so that we don't accidentally break the system.